

Actus 记忆系统：把长期记忆做成一套可运行、可治理的 Agent 基础设施

摘要 (官网预览)

很多 Agent 的问题，并不是模型不够强，而是记忆能力不够完整。它们能回答当前问题，却难以在跨会话协作中稳定保留用户偏好、团队规则和长期事实；能临时引用上下文，却很难把这些信息沉淀为可检索、可治理、可审计的系统资产。Actus 想解决的，正是这一层。

[配图占位符 01：封面图 (Actus + Memory Infrastructure + Long-term Collaboration)]

很多团队在评估 Agent 时，最先关注的通常是：

- 会不会调用工具
- 会不会拆解任务
- 会不会写代码、查资料、跑流程

但当 Agent 真正进入业务系统后，决定可用性的往往不是这些前台能力，而是一个更底层的问题：

它能不能把真正重要的信息稳定沉淀下来，并在后续协作里以正确的方式重新取用。

这是 Agent 从“单轮回答”走向“持续协作”时绕不过去的一步。

因为真实业务里的协作，不是一次性问答，而是不断累积的过程：

- 用户会反复回来继续同一个任务
- 团队会逐步形成稳定的输出偏好和流程约束
- 项目会沉淀越来越多后续仍然有价值的事实

如果 Agent 每次都像第一次认识用户，
那它即使短期表现不错，也很难真正成为稳定的生产工具。

所以，记忆系统不是 Agent 的附加模块，
而是它能否进入长期工作流的底层基础设施。

[配图占位符 02：从“单轮对话”到“持续协作”的能力跃迁示意图]

一句话理解 Actus 记忆系统

Actus 对“记忆”的理解，不是拉长聊天历史，也不是把所有内容都塞回上下文。

它要做的是把对后续协作真正有价值的信息，沉淀成可分类、可检索、可治理、可审计的长期记忆资产。

这意味着系统要同时回答五个问题：

- 什么内容值得记
- 记下来之后属于哪一类
- 后续应该在什么时机被召回
- 这些记忆如何与运行时上下文协同
- 当系统进入生产环境后，这些记忆如何治理、修复和审计

从这个角度看，Actus 记忆系统不是一个“备忘录功能”，而是一套围绕长期协作构建的系统能力。

[配图占位符 03：Actus 记忆系统全景图（对象模型 / 写入 / 召回 / 运行语义 / 治理）]

1) 先解决“记什么”：把长期记忆做成有结构的对象模型

很多产品谈记忆，本质上还是在延长消息历史。

这种方式在短会话里可能足够，但在长期协作里，很容易把高价值信息和短期噪音混在一起。

Actus 先做的，是把长期记忆建模成明确的对象，而不是一段模糊的聊天残留。

在当前设计里，长期记忆至少会被区分为三类：

- **用户型记忆 (user)**：例如角色背景、沟通风格、偏好设置、工作习惯
- **规则型记忆 (rule)**：例如团队规范、固定流程、长期约束、输出要求
- **事实型记忆 (fact)**：例如项目背景、业务结论、稳定事实、后续可复用信息

这种分类不是为了“看起来更整齐”，而是为了让后续系统行为更可控：

- 用户型记忆回答“这个人是谁、偏好是什么”
- 规则型记忆回答“这个任务应该怎么做、需要遵守什么”
- 事实型记忆回答“哪些信息以后还会被反复用到”

在对象层面，Actus 也会给每条记忆附带必要的元信息，例如：

- category
- source
- tags
- pinned

- created_at / updated_at
- auto_promoted_at

这让记忆从一段“模型好像知道的东西”，变成一类能被系统理解和管理的数据对象。

[配图占位符 04 : user / rule / fact 记忆对象模型示意图]

2) 再解决“怎么记”：显式写入、自动沉淀与质量闸是同一条链路

如果一个 Agent 把所有对话都写成长期记忆，它得到的通常不是更强的能力，而是一套越来越嘈杂、越来越不可控的历史垃圾堆。

所以，Actus 在写入链路上的重点，不是“能写进去”，而是“如何克制地写进去”。

从当前系统设计看，长期记忆主要来自两类入口：

- **显式写入**：用户或 Agent 主动调用 memory_save，把明确重要的信息沉淀为长期记忆
- **自动沉淀**：会话过程中的内容经过 session flush、总结和筛选后，进入长期记忆链路

这两类入口最终都不会绕开治理逻辑。

Actus 在写入链路里补上了几层关键能力：

- **质量闸 (memory gate)**：自动沉淀不是“看到就记”，而是先做质量判断
- **配额控制**：限制长期记忆写入速率和规模，避免无边界膨胀
- **会话级限制**：防止 Agent 在单个 session 里频繁调用 memory_save
- **通知机制**：当 gate 暂停、配额超限或文件同步异常时，系统能够显式发出信号

这条设计的价值在于，Actus 并不假设“信息越多越好”。

它更关心的是：

- 这条信息是否值得长期保留
- 这次写入是否符合当前系统边界
- 当自动机制出现异常时，团队是否能及时感知

这也是为什么 Actus 的记忆写入，更像一条可治理的数据写入链路，而不是一个静默堆积的黑盒。

[配图占位符 05 : 记忆写入链路图 (memory_save / session flush -> gate -> persist)]

3) 真正难的不是“写进去”，而是“取对”：召回与排序决定记忆是否可用

长期记忆的真正难点，从来都不只是落盘，
而是后续任务发生时，系统能不能把对的内容、在对的时机、以对的粒度拿出来。

Actus 在这条链路上做的，不是简单关键词匹配，
而是更接近业务可用性的混合检索思路。

当前系统中，记忆检索更接近这样一条逻辑：

- 先基于语义相似度做初步召回
- 再叠加时间衰减，让较新的相关记忆优先
- 再结合标签和类别做进一步排序补强

它背后的出发点很明确：

- 用户表达的是相近语义，不是完全相同的字面，也应该被召回
- 新近且相关的记忆，通常比很久以前的旧记忆更有价值
- 已经分类和打标签的记忆，应该在检索上具备更强可解释性

对业务团队来说，这带来的直接效果是：

- Agent 不只是“曾经记住过”
- 而是更有机会在真正执行任务时“想得起来”

对技术负责人来说，这意味着记忆系统不再依赖“把所有历史塞回 prompt”这种粗放做法，
而是具备了更清晰的召回和排序机制。

在运行上，这也让系统可以把“搜索”与“读取完整内容”拆开：

先召回候选，再按需读取完整记忆，而不是一开始就把所有内容推给模型。

这一步对控制上下文噪音非常关键。

[配图占位符 06：记忆召回与排序示意图（语义相关 / 时间权重 / 标签增强）]

4) 记忆不是只存在一个地方：Actus 用三视图定义运行语义

很多 Agent 产品在谈记忆时，会默认它只是 prompt 的一部分。

但一旦系统进入真实运行环境，这种理解就不够了。

Actus 在记忆系统里明确区分了三个视图：

- **DB 视图**：数据库中的权威检索视图

- **File 视图**：落到宿主机文件系统中的事实源视图
- **Prompt 视图**：session 启动时注入模型上下文的快照视图

这不是实现细节，而是系统语义。

DB 视图：检索权威

memory_search / memory_recall 这类能力首先依赖数据库视图。

它的特点是：

- 写入提交后即可检索
- 适合排序、过滤、管理和审计
- 是系统判断“有哪些记忆存在”的主要入口

File 视图：事实源与可维护载体

Actus 同时把记忆规范化落盘为 Markdown 文件。

这带来的价值不是“多存一份”，而是让记忆开始具备：

- 可浏览
- 可版本化
- 可手工修正
- 可与外部文件系统工作流衔接

更关键的是，File 视图通过 bind mount 进入 sandbox 时是 **LIVE view**：

宿主机上的写入会实时反映到运行中的沙箱里，而不是等待重启。

Prompt 视图：session-scoped snapshot

模型实际看到的记忆，不是完整数据库，也不是整个文件系统，而是 session 启动时装配出来的 prompt 快照。

这意味着：

- user / rule 类记忆会在会话启动时注入
- fact 类记忆更适合以精简索引形式进入 prompt
- session 进行中的新增记忆，不会自动改写当前 prompt

这个“不自动同步”不是 bug，而是明确的系统边界。

Prompt 是会话级快照，而不是随时重建的共享状态。

对技术团队来说，这种三视图设计非常重要，因为它把不同能力的职责拆开了：

- 检索依赖 DB
- 文件可维护性依赖 File
- 模型推理依赖 Prompt

系统不再假设“一份状态解决所有问题”，
而是承认不同场景需要不同视图。

[配图占位符 07：DB / File / Prompt 三视图运行语义图]

5) 三视图不是天然一致的，所以需要显式的一致性策略

当记忆同时存在 DB、文件和 prompt 三个视图中时，
一致性问题就不再是边缘情况，而是设计中心。

Actus 在这里没有回避复杂性，而是把差异显式定义出来。

例如：

- API 写入后，DB 可立即检索
- 文件会异步落盘，但进入稳定状态后可在 File 视图看到
- 当前 session 的 prompt 不会因为中途写入而即时重组
- 用户手工修改文件时，File 视图能立刻看到，但 DB 需要通过 reconcile 才会对齐

这意味着 Actus 选择的不是“假装完全实时一致”，
而是把各视图的时效性和边界明确下来，再用同步修复机制兜住。

这里最关键的能力是 **reconciler**：

- 可以修补 DB 与文件之间的落盘 backlog
- 可以在用户重新进入 session 时扫描并处理不一致
- 可以把异常文件隔离到 .orphans/
- 可以在必要时重建缺失的文件视图

这套设计对技术负责人有一个很实际的意义：

系统不是只在理想路径上工作，而是为真实环境里的漂移、补写、人工干预和失败恢复预留了机制。

[配图占位符 08：三视图不一致与 reconcile 修复示意图]

6) 为什么这套记忆系统更适合进入生产环境

对官网文章来说，真正值得讲清楚的，不只是“Actus 有记忆”，而是“为什么这套记忆系统适合进入生产环境”。

这背后至少有五层支撑。

1. 写入链路是可治理的

长期记忆不是一个无边界增长的容器。

Actus 通过质量闸、配额和会话级限制，把写入控制在可观察、可约束的范围内。

2. 持久化不是脆弱的单点动作

文件写入不是一次简单覆盖，而是围绕原子落盘、失败重试和状态标记来设计。

这让“记忆已写入”不只是一句乐观假设，而是一个可以追踪的过程。

3. 异常可以被系统感知

当质量闸暂停、配额超限、文件同步长期失败时，

系统可以通过通知链路把这些异常显式暴露出来，而不是静默退化。

4. 记忆具备审计和运维抓手

写入来源、分类、更新时间、自动晋升时间等元信息，

让团队可以回答很多生产环境下必须回答的问题：

- 这条记忆是人工写入还是自动沉淀
- 它是什么时候进入系统的
- 最近有没有被更新
- 哪些自动沉淀值得回看和审查

5. 运行时边界是被认真处理过的

记忆进入 sandbox 时并不是裸暴露。

Actus 在这里强调的是只读挂载、安全边界和纵深防御，例如：

- sandbox 内对 memory mount 的写入会被内核级只读限制拦住
- 写入侧会拒绝 symlink
- reconciler 会隔离宿主机侧的异常链接和异常文件

这些能力看上去不如“会不会自动记住用户偏好”那么显眼，但它们才真正决定了这套系统能不能放心进入生产。

[配图占位符 09：生产级治理能力图 (quota / audit / reconcile / notifications / security)]

对业务团队的价值：让 Agent 从“会回答”走向“会持续协作”

当记忆系统被做成一套完整基础设施后，业务团队得到的变化，不只是回复更像“记得你”，而是协作关系本身变得连续了。

几个最直接的价值会很明显：

- **跨会话连续性**：任务不必每次从零开始
- **长期偏好沉淀**：输出形式、表达风格、协作习惯可以稳定下来
- **规则持续生效**：团队约束和固定流程不需要反复重复
- **复杂任务更稳**：关键事实不会轻易被长上下文噪音淹没

对业务负责人来说，这意味着 Agent 更像一个会逐步熟悉团队和任务的长期协作者，而不是一个每次都重新培训的临时工具。

[配图占位符 10：业务价值场景图 (知识助手 / 运营协作 / 研发支持 / 内部平台)]

对开发者和技术负责人的价值：把“记忆”从 Prompt 技巧升级为系统能力

如果你是技术负责人，你真正会问的通常不是“它像不像记住了”，而是：

- 这套记忆怎么建模
- 怎么写入
- 怎么召回
- 怎么保持边界清晰
- 怎么处理异常与不一致
- 怎么在生产环境里审计和修复

从这个角度看，Actus 记忆系统更值得关注的地方在于：

- 它不是单纯延长上下文
- 它不是简单保存聊天历史
- 它不是把所有记忆都交给模型临场发挥

它在往一套更完整的基础设施演进：

- 有对象模型
- 有写入链路
- 有召回排序
- 有三视图运行语义
- 有一致性修复和治理机制
- 有进入 sandbox 的安全边界

这会让技术团队更容易判断：

它不是一个“看起来有记忆”的演示能力，而是一套可以真正接进产品系统的长期记忆架构。

[配图占位符 11：技术负责人视角总结图（模型 / 链路 / 召回 / 一致性 / 治理）]

结语

如果把 Agent 理解成一次性问答工具，
那么记忆系统确实像一个加分项。

但如果你关心的是：

- 它能不能跨天持续协作
- 能不能逐步理解用户和团队
- 能不能在复杂流程里稳定保留关键事实
- 能不能把长期经验沉淀成真正可管理的系统资产

那么记忆系统就不是外挂能力，
而是 Agent 产品化过程中的核心基础设施。

Actus 正在补齐的，正是这一层。

它不是要让 Agent “多背一点”，
而是要让 Agent 在真实业务里，具备长期协作所需要的记忆能力、运行语义和治理边界。